

Types as expressions

Dennis Korpel

September 26, 2020, Reimplementing the bulk of std.meta iteratively, Andrei Alexandrescu

“ The algorithms follow the following pattern consistently:

1. Reify the compile-time parameters into values
2. Carry processing on these values with the usual algorithms
3. If needed, dereify back the results into compile-time parameters

”

Note: not talking about type manipulation.

Source: [https://forum.dlang.org/thread/rknpkj\\$d7f\\$1@digitalmars.com?page=1](https://forum.dlang.org/thread/rknpkj$d7f$1@digitalmars.com?page=1)

What Andrei used:

1. Convert type to string with `.stringof`
2. Pass string variables around
3. Convert back with `mixin()`

What you can do today:

1. Convert type to `string` with `.mangleof`
2. Pass string variables around
3. Convert back with `__traits(toType, stringValue)`

Example:

```
string unsignedOfSize(size_t n)
{
    if (n == 4)
        return uint.mangleof;
    if (n == 2)
        return ushort.mangleof;
    if (n == 1)
        return ubyte.mangleof;
    assert(0);
}

__traits(toType, unsignedOfSize(4)) myInteger;
```

Works today:

```
alias ToType(string val) = __traits(toType, val);  
  
ToType!(unsignedOfSize(4)) myInteger;
```

Idea: types in expressions are

`type_t`

```
type_t myInt = int;  
type_t[3] floats = [float, double, real];
```


Works: Static array parsing

```
auto foo(alias X, alias Y)()
{
    return X[Y].init;
}
```

```
alias f0 = foo!(int, 3); // int[3].init = [0, 0, 0]
char f1 = foo!("abc", 3); // "abc"[3].init = char.init = 0xFF
```

Requires parser change: Pointer syntax, type constructor

```
auto toPtr(type_t T)
{
    return T*;
}

type_t addConst(type_t t)
{
    return const(t);
}
```

Works: optional is()

```
// Current:  
static if (is(typeof(a) == int))  
{  
  
}  
  
// Now also allowed:  
static if (typeof(a) == int)  
{  
  
}
```

Works: Switch

```
type_t unsignedOf(type_t a)
{
    switch (a)
    {
        case int: return uint;
        case short: return ushort;
        case long: return ulong;
        default:
            assert(0);
    }
}
```

Works: Associative Arrays

```
enum toUnsigned = [  
    int: uint,  
    short: ushort,  
    long: ulong  
];  
static assert(toUnsigned[int] == uint);  
static assert(toUnsigned[short] == ushort);
```

Complete example:

std/traits.d:

```
template AllImplicitConversionTargets(T)
{
    static if (is(T == bool))
        alias AllImplicitConversionTargets =
            AliasSeq!(byte, AllImplicitConversionTargets!byte);
    else static if (is(T == byte))
        alias AllImplicitConversionTargets =
            AliasSeq!(char, ubyte, short, AllImplicitConversionTargets!short);
    else static if (is(T == ubyte))
        alias AllImplicitConversionTargets =
            AliasSeq!(byte, char, short, AllImplicitConversionTargets!short);
    else static if (is(T == short))
        alias AllImplicitConversionTargets =
            AliasSeq!(ushort, wchar, int, AllImplicitConversionTargets!int);
    else static if (is(T == ushort))
        alias AllImplicitConversionTargets =
            AliasSeq!(short, wchar, dchar, AllImplicitConversionTargets!dchar);
    else static if (is(T == int))
        alias AllImplicitConversionTargets =
            AliasSeq!(dchar, uint, long, AllImplicitConversionTargets!long);
    else static if (is(T == uint))
        alias AllImplicitConversionTargets =
            AliasSeq!(dchar, int, long, AllImplicitConversionTargets!long);
    else static if (is(T == long))
        alias AllImplicitConversionTargets = AliasSeq!(ulong, float, double, real);
    else static if (is(T == ulong))
        alias AllImplicitConversionTargets = AliasSeq!(long, float, double, real);
    else static if (is(T == float))
        alias AllImplicitConversionTargets = AliasSeq!(double, real);
    else static if (is(T == double))
        alias AllImplicitConversionTargets = AliasSeq!(float, real);
    else static if (is(T == real))
        alias AllImplicitConversionTargets = AliasSeq!(float, double);
    else static if (is(T == char))
        alias AllImplicitConversionTargets =
            AliasSeq!(byte, ubyte, short, AllImplicitConversionTargets!short);
    else static if (is(T == wchar))
        alias AllImplicitConversionTargets =
            AliasSeq!(short, ushort, dchar, AllImplicitConversionTargets!dchar);
    else static if (is(T == dchar))
        alias AllImplicitConversionTargets =
            AliasSeq!(int, uint, long, AllImplicitConversionTargets!long);
    else static if (is(T == class))
        alias AllImplicitConversionTargets = staticMap!(ApplyLeft!(CopyConstness, T), TransitiveBaseTypeTuple!T);
    else static if (is(T == interface))
        alias AllImplicitConversionTargets = staticMap!(ApplyLeft!(CopyConstness, T), InterfacesTuple!T);
    else static if (isArray!T && !is(typeof(T.init[0]) == const))
    {
        static if (is(typeof(T.init[0]) == shared))
            alias AllImplicitConversionTargets =
                AliasSeq!(const(shared(Unqual!(typeof(T.init[0]))))[]);
        else
            alias AllImplicitConversionTargets =
                AliasSeq!(const(Unqual!(typeof(T.init[0]))))[];
    }
    else static if (is(T : void*) && !is(T == void*))
        alias AllImplicitConversionTargets = AliasSeq!(void*);
    else
        alias AllImplicitConversionTargets = AliasSeq!();
}
```

After:

```
type_t[] allImplicitConversionTargets(type_t T)
{
    switch (T)
    {
        case bool:    return [byte] ~ allImplicitConversionTargets(byte);
        case byte:    return [char, ubyte, short] ~ allImplicitConversionTargets(short);
        case ubyte:    return [byte, char, short] ~ allImplicitConversionTargets(short);
        case short:    return [ushort, wchar, int] ~ allImplicitConversionTargets(int);
        case ushort:   return [short, wchar, dchar] ~ allImplicitConversionTargets(dchar);
        case int:      return [dchar, uint, long] ~ allImplicitConversionTargets(long);
        case uint:     return [dchar, int, long] ~ allImplicitConversionTargets(long);
        case long:     return [ulong, float, double, real];
        case ulong:    return [long, float, double, real];
        case float:    return [double, real];
        case double:   return [float, real];
        case real:     return [float, double];
        case char:     return [byte, ubyte, short] ~ allImplicitConversionTargets(short);
        case wchar:    return [short, ushort, dchar] ~ allImplicitConversionTargets(dchar);
        case dchar:    return [int, uint, long] ~ allImplicitConversionTargets(long);
        default:
            if (is(T == class))
                return transitiveBaseTypeTuple(T).map!(B => copyConstness(T, B)).array;
            else if (is(T == interface))
                return interfacesTuple(T).map!(I => copyConstness(T, I)).array;
            else if (isDynamicArray(T) && !is(typeof(T.init[0]) == const))
            {
                if (is(typeof(T.init[0]) == shared))
                    return [const(shared(unqual(typeof(T.init[0]))))][];
                else
                    return [const(unqual(typeof(T.init[0])))][];
            }
            else if (is(T : void*) && !is(T == void*))
                return [void*];
            else
                return [];
    }
}
```

What doesn't (always) work

- variable's value vs type conflation

```
static_assert(3.sizeof == int.sizeof);
```

```
type_t largerType(type_t a, type_t b)
{
    return a.sizeof > b.sizeof;
}
```

```
enum staticArraySize(alias T, size_t length) = T.sizeof * length;
```